Python Activity-Based Learning
Case Study

42 Student Marks Matrix Analysis

Selected case study | Prepared using the topics listed in the supplied assignment.

1. Problem Statement

Store marks of multiple students across multiple subjects in a nested list. Calculate each student's total and average, subject averages, and identify the top student.

2. Objectives

• Represent tabular data using nested lists.

• Use nested loops to process rows and columns.

• Calculate row-wise and column-wise totals and averages.

• Identify the highest-performing student.

3. Assumptions

• There are three students and three subjects.

• Each mark is an integer from 0 to 100.

• Student names are stored in a separate list.

• The highest total identifies the top student.

4. Algorithm

1. Read three student names and their marks in three subjects.

2. Store marks in a nested list.

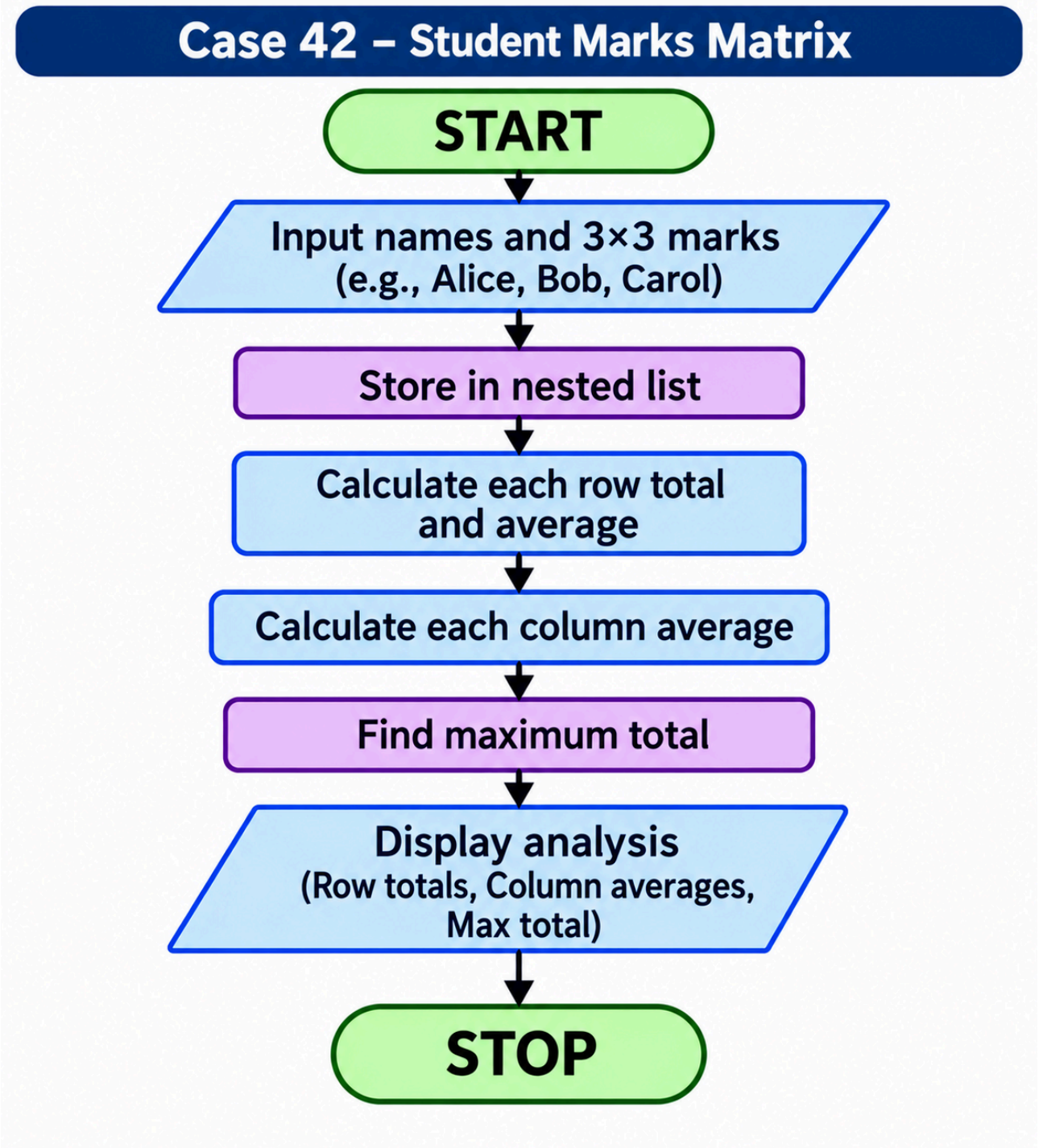
3. For each student, calculate total and average.

4. For each subject, calculate the average across students.

5. Find the student with the highest total.

6. Display all results.

5. Flowchart



6. Pseudocode

```
BEGIN
 INPUT 3 student names and 3 marks each
 FOR each student row
 total ← sum of row
 average ← total / 3
 STORE total
 END FOR
 FOR each subject column
 subject_total ← sum marks in column
```

```
 PRINT subject_total / 3
 END FOR
 Find index of highest student total
 PRINT student results and topper
END
```

Case Study 42 — Student Marks Matrix Analysis

7. Python Source Code

```
names = []
marks = []

for i in range(3):
 name = input("Student name: ")
 names.append(name)
 row = []
 for j in range(3):
 mark = int(input("Mark in subject " + str(j + 1) + ": "))
 row.append(mark)
 marks.append(row)

totals = []
print("\n--- STUDENT RESULTS ---")
for i in range(3):
 total = sum(marks[i])
 average = total / 3
 totals.append(total)
 print(names[i], "Marks:", marks[i],
 "Total:", total, "Average:", round(average, 2))

print("\n--- SUBJECT AVERAGES ---")
for j in range(3):
 subject_total = 0
 for i in range(3):
 subject_total += marks[i][j]
 print("Subject", j + 1, "Average:", subject_total / 3)

top_index = totals.index(max(totals))
print("\nTop student:", names[top_index],
 "with total", totals[top_index])
```

8. Test Cases

Input / Scenario	Expected result
A: 80,70,90; B: 60,75,65; C: 90,85,95	Totals 240, 200, 270; top student C
All students score 50 in all subjects	All totals 150; first student is displayed as top due to tie handling
Student mark 110	Outside assumed valid range; input should be corrected

9. Conclusion

The program demonstrates nested lists and nested loops for analysing student marks by student and by subject. Note: Run the program in Python 3. The sample test cases are illustrative; enter the values when prompted.